ConnectHub: An Offline-First Android Blogging Application

Supporting Documentation

Module: 6CS027 Secure Mobile Application Development

Assessment: Portfolio (coursework artefact and supporting documentation)

Academic year: 2025/2026, Semester 1

Student name: [Student name]

Student ID: [Student ID]

1. Introduction and Objectives

ConnectHub is a native Android application, written in Java with XML layouts, for writing and managing blog posts. Because mobile connectivity is unreliable, the device is the primary store: every post is saved in a local SQLite database, and the internet is needed only when the user chooses to publish. The objectives were to provide complete offline post management, attach photos from the gallery and camera, share posts through Android intents, and publish selected posts to a personal Blogger blog.

2. Requirements and Features

The implemented features are:

- Create, edit, view and delete posts; a list screen with status filters; a detail screen for each post.
- Search across titles and bodies (partial, case-insensitive), listing every match with highlighting.
- Single deletion with confirmation; multi-selection with select-all, clear and bulk deletion.
- SQLite storage, including drafts introduced in schema version 2.
- Photo attachment from the system Photo Picker or the device camera.
- Sharing by email or any compatible app through the standard ACTION_SEND intent.
- Uploading one or several posts to Blogger through the Blogger API v3, plus a clearly labelled Test mode.

Real Blogger publishing is implemented but has not yet been verified, because it requires an OAuth client registered to the signing certificate (Section 6).

3. Software Design

MainActivity hosts five bottom-navigation tabs (Home, Posts, Upload, Gallery, Profile); separate activities provide search, editing, post detail and image preview. Figure 1 shows the principal use cases.

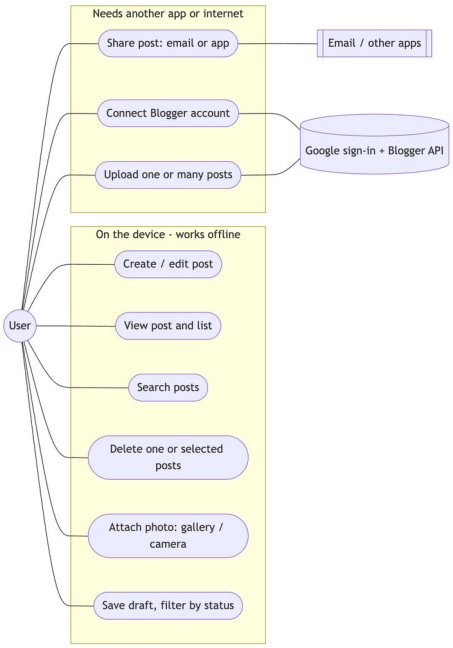


Figure 1. Use-case diagram

The code is layered (Figure 2). Activities and fragments handle presentation only. **PostRepository** is the only class that executes SQL, through **BlogDbHelper** (SQLiteOpenHelper). **ImageStorage** manages photo files and **ShareHelper** builds share intents. **PublishController** coordinates confirmation, authorisation and uploads through the **PublishService** interface, implemented by BloggerPublishService and MockPublishService, so simulated runs can never mark a post as published. Database, file and network work runs on background threads.

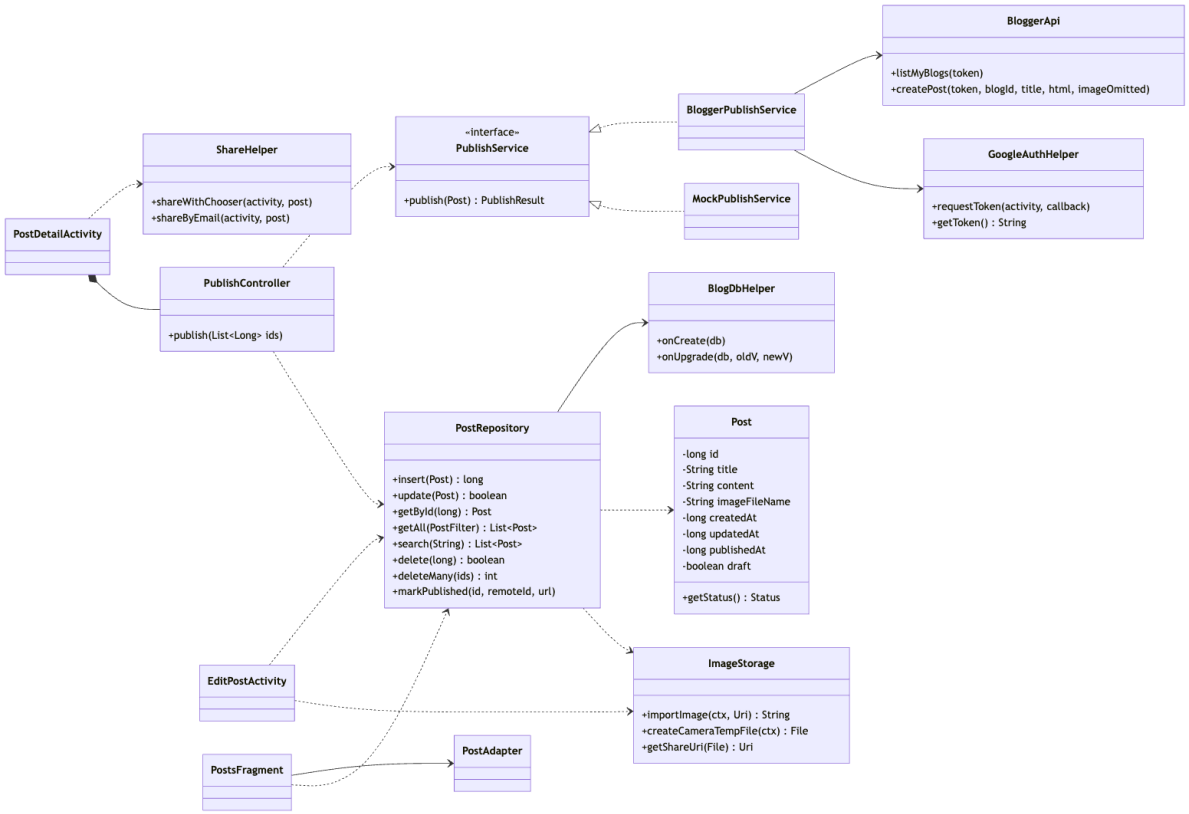


Figure 2. Core class diagram (full diagram in software-design/)

4. Database Design and Persistence

The database `blogspace.db` contains one table, `posts` (Figure 3), keyed by an auto-increment `_id`. Title and content are mandatory TEXT fields; timestamps are stored as epoch milliseconds; `remote_post_id`, `remote_url` and `published_at` record confirmed uploads; `is_draft` was added in version 2 by `onUpgrade`, which alters the table rather than dropping it.

Photos are not stored in SQLite. Each selected or captured image is decoded, rotated upright, scaled to at most 2048 px and saved as a JPEG in app-private storage; the table stores only its file name. Records survive restarts because SQLite writes to internal storage. Bulk deletion runs in one transaction, and photo files are removed only after it commits.

POSTS			
INTEGER	_id	PK	AUTOINCREMENT
TEXT	title		NOT NULL
TEXT	content		NOT NULL
TEXT	image_file		NULL - file name in filesDir/images
INTEGER	created_at		NOT NULL, epoch ms
INTEGER	updated_at		NOT NULL, epoch ms, indexed DESC
TEXT	remote_post_id		NULL - Blogger id after confirmed publish
TEXT	remote_url		NULL
INTEGER	published_at		NOT NULL DEFAULT 0
INTEGER	is_draft		NOT NULL DEFAULT 0 (added in v2)

Figure 3. posts table

5. Implementation and Key Decisions

- **Validation:** titles and bodies are required, trimmed, and limited to 120 and 20,000 characters; drafts need either field.
- **Security:** all queries use bound parameters and escape LIKE wildcards. No camera or storage permission is requested: the gallery uses the Photo Picker, the camera app writes into a FileProvider URI, and shared photos receive only temporary read grants.
- **Publishing:** Google Play services AuthorizationClient provides an OAuth 2.0 token; the Android client is identified by package name and SHA-1, so no password or secret is stored. A post is marked Uploaded only after Blogger returns its id.
- **Third-party code:** AndroidX, Material Components, Google Play services Auth and the Inter typeface (SIL Open Font License).

Three challenges arose. Gallery URIs and temporary camera files are not durable, so images are copied into app storage. The Gallery screen crashed because a grid layout animation is incompatible with RecyclerView; emulator testing exposed this and it was replaced. Target SDK 36 enforces edge-to-edge drawing, handled by applying window-inset padding. Figures 4 and 5 show the resulting screens.

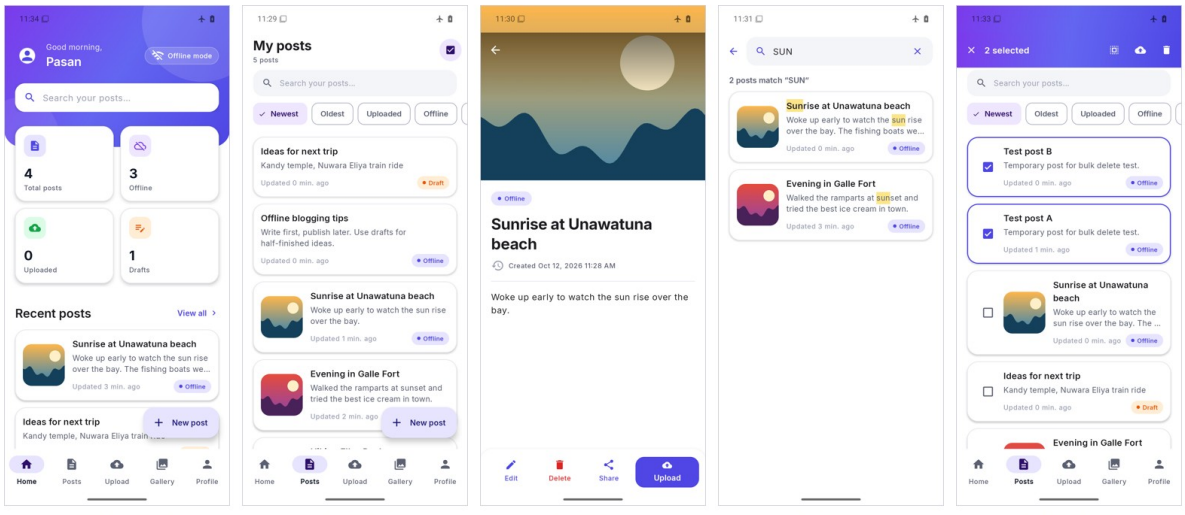
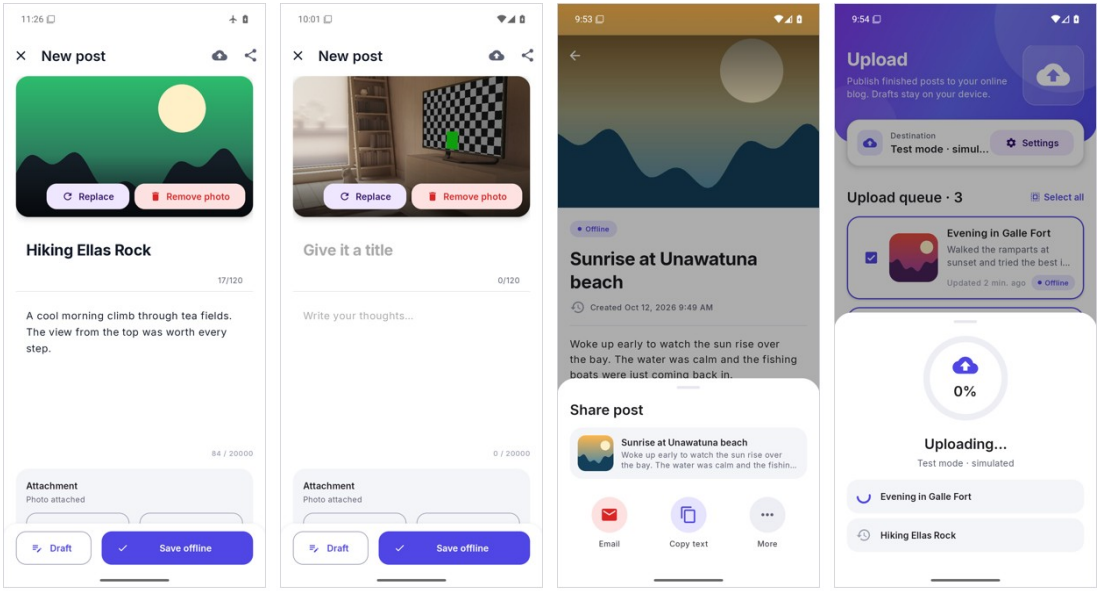


Figure 4. Core screens captured in airplane mode



(a) Gallery photo (b) Camera photo (c) Share sheet (d) Upload (Test mode)

Figure 5. Photo attachment, sharing and upload (Test mode)

6. Testing and Results

Testing combined 24 JVM unit tests and 23 instrumented tests on an Android 16 emulator (all passed) with manual tests performed in airplane mode and recorded as screenshots. Full cases are in test-plan-and-results.md.

Table 1. Testing summary

Area	Evidence	Result
Create, edit, view, delete	Manual (airplane mode); instrumented	Passed
Search, bulk deletion	Manual; instrumented	Passed
Persistence after restart	Force-stop and relaunch; instrumented	Passed
Gallery photos (JPEG, PNG, WebP)	Manual; instrumented	Passed
Camera capture	Emulator virtual camera	Passed (no physical device)
Sharing	Intent contents tested; share sheet opened	Partial: email delivery not checked
Upload flow	Test mode only	Passed (simulated)
Real Blogger publishing	None yet	Not tested
Validation and errors	Manual; unit	Partial: upload-failure path not run

7. Limitations and Future Work

Real publishing depends on one-time Google Cloud configuration and remains unverified. The Blogger API has no media endpoint, so photos are embedded as reduced data URIs; if Blogger rejects one, the post is resent without it. Testing used one emulator profile. Future work includes verifying publishing with a configured account, testing on physical devices and adding database backup.

8. Conclusion

ConnectHub implements offline post management, SQLite persistence, gallery and camera attachments, intent-based sharing and a Blogger publishing workflow. Automated and manual tests confirm the offline features, storage and image handling. Email delivery and real online publishing must still be demonstrated with configured accounts before every criterion can be considered met.

References

Android Developers (n.d.) *Save data using SQLite*. Available at: https://developer.android.com/training/data-storage/sqlite (Accessed: 12 October 2026).

Android Developers (n.d.) *Photo picker*. Available at: https://developer.android.com/training/data-storage/shared/photo-picker (Accessed: 12 October 2026).

Android Developers (n.d.) *Camera intents*. Available at: https://developer.android.com/training/camera/camera-intents (Accessed: 12 October 2026).

Android Developers (n.d.) *FileProvider*. Available at: https://developer.android.com/reference/androidx/core/content/FileProvider (Accessed: 12 October 2026).

Android Developers (n.d.) *Send data to other apps*. Available at: https://developer.android.com/training/sharing/send (Accessed: 12 October 2026).

Android Developers (n.d.) *Authorize access to Google user data*. Available at: https://developer.android.com/identity/authorization (Accessed: 12 October 2026).

Google for Developers (n.d.) *Blogger API: Using the API*. Available at: https://developers.google.com/blogger/docs/3.0/using (Accessed: 12 October 2026).